

ELECTRONIC ORDER BOOK

Engineering and Performance Report

Matching • Replay • Risk • P&L • UDP Recovery • Scalability

400,391

LOBSTER-derived source rows

10 million

maximum event applications

2.17-2.25m/s

median direct replay throughput

0 misses

UDP recovery failures

C++20 | Windows | CMake | MinGW | Python analysis

1. Problem statement and objective

Electronic trading infrastructure must process large, ordered streams of market events while preserving an economically correct representation of the market. Raw speed alone is not sufficient: a system can continue processing while its order book is corrupted by a missed update, its order index diverges from queued state, a strategy exceeds its permitted exposure, or downstream cash, inventory and P&L no longer reconcile with executed trades.

The objective of this project is to design and validate a modular C++20 electronic order-book and matching-engine simulator that treats performance, correctness, financial controls and recovery as connected engineering requirements rather than separate demonstrations.

The system is evaluated through deterministic replay of 400,391 LOBSTER-derived AAPL source rows. It combines price-time-priority matching with normalised event replay, binary event handling, market-making strategy integration, configurable pre-trade risk controls, position and P&L accounting, structured telemetry, and a sequenced UDP-style feed capable of detecting and recovering deliberately omitted updates.

Central engineering question

Can the engine process large, stateful market-event workloads at stable latency while preserving price-time priority and order-book invariants, maintaining coherent risk and accounting state, and recovering to the same final state as a clean replay after market-data loss?

The project is best understood as a small electronic-trading infrastructure stack. It is not presented as a production exchange or as an end-to-end wire-latency claim; it is a controlled engineering platform for testing the interaction between performance, state integrity, financial controls and failure recovery.

1.1 What this project does differently

Real event-flow input - The engine is evaluated with approximately 400,000 LOBSTER-derived source rows rather than relying only on hand-written or uniformly random synthetic orders.

Performance and correctness measured together - The benchmark records runtime, throughput, sampled p50/p95/p99 latency and memory, while also requiring zero rejected events and successful order-book invariant checks.

Deliberate failure injection - The UDP benchmark intentionally removes sequenced updates, verifies that gaps are detected, recovers the missing events and checks that recovery misses remain at zero.

Separate fast path and recovery path - Direct replay isolates event-application performance, while UDP scalability measures the additional cost of serialisation, queueing, decoding, sequencing and recovery.

Broader trading-system scope - The repository combines matching, deterministic replay, binary codecs, market-data recovery, risk controls, position/P&L processing, telemetry and Python analysis.

Transparent data limitations - The report explicitly distinguishes a useful LOBSTER-derived event stream from a complete original-market L3 reconstruction.

1.2 Engineering depth behind these differences

Stateful market-data reconstruction - The engine is evaluated with 400,391 LOBSTER-derived AAPL source rows. The loader must interpret stateful order messages, maintain active order IDs, handle references outside the locally available state and distinguish visible from hidden executions. This is materially harder than replaying independent random orders.

Correctness-gated performance - A run is only accepted when benchmark events are applied without rejection and the book passes structural invariant checks. Throughput and tail latency are therefore reported only alongside evidence that the internal state remains valid.

Fault injection with state equivalence - The UDP workflow deliberately omits sequenced updates, detects the gap, recovers the missing event and restores ordered processing. The full recovery test compares the recovered final book with a clean loss-free replay, validating the resulting state rather than merely counting a recovery attempt.

Separate fast and resilient paths - Direct replay isolates event-application performance. UDP scalability measures the additional work from serialisation, queueing, decoding, sequence validation and recovery, making the cost of resilience visible.

Integrated trading-system responsibilities - Matching, replay, binary protocols, strategy logic, risk controls, trade processing, inventory, P&L, telemetry and Python analysis are connected in one repository. This is broader than implementing add, cancel and match operations in isolation.

Measurement hygiene - Input conversion, telemetry writing and post-batch validation are kept outside the direct timed region. Latency is sampled and containers are prepared before timing where possible, reducing measurement contamination.

Transparent data limitations - Missing references and hidden executions are counted rather than concealed or fabricated. The report limits its claims to event-flow, correctness and scalability instead of misrepresenting the result as a complete original-market L3 reconstruction.

2. Executive findings

The direct replay benchmark used 380,678 converted events derived from approximately 400,000 source rows. It tested five workload sizes from 100,000 to 10 million event applications, with five repetitions at each size.

- 25 of 25 direct-replay runs completed successfully.
- Zero benchmark events were rejected.
- All order-book invariant checks passed.
- Median direct-replay throughput remained approximately 2.16-2.25 million events per second.
- The median 10-million-event runtime was 4.601 seconds.
- Sampled p99 direct-event latency remained approximately 0.9-1.0 microseconds.
- Median peak working-set memory increased by only approximately 1.9 MB from 100,000 to 10 million applications.
- Every deliberately omitted UDP event was recovered, with zero recovery misses.

Workload	Median runtime	Median throughput	Median p99	Peak memory
100,000	0.046 s	2,160,354/s	1,000 ns	35.312 MB
500,000	0.223 s	2,246,922/s	900 ns	36.137 MB
1,000,000	0.449 s	2,226,734/s	900 ns	36.297 MB
5,000,000	2.242 s	2,229,836/s	900 ns	36.867 MB
10,000,000	4.601 s	2,173,241/s	1,000 ns	37.215 MB

Interpretation boundary

These are Debug-build, application-level measurements on a general-purpose Windows machine. They are evidence of scaling and correctness in the tested environment, not a production exchange latency claim.

3. System scope

3.1 Matching engine and order-book state

The core engine maintains independent bid and ask sides, ordered by economically correct price priority. Orders sharing a price level are processed in FIFO order. An order index supports direct lookup by order ID, while invariant validation checks that price levels, queued orders and the index remain mutually consistent.

The matching tests cover order entry, cancellation, modification, aggressive execution, partial fills, multiple fills and removal of completed price levels. Functional correctness is established before performance measurements are accepted.

3.2 Deterministic replay and binary event handling

Market-data messages are normalised into a common MarketEvent representation. Events can be serialised into a binary format, parsed back and compared with the original vector. A changed or corrupted round trip causes the replay validation to fail.

Determinism is relevant to trading infrastructure because the same ordered input should reproduce the same state during backtesting, audit and incident reconstruction.

3.3 Strategy and risk controls

The project includes a market-making strategy that observes current market state and attempts to refresh bid and ask quotes. Candidate quote generation is separated from configured risk approval. Strategy and risk tests exercise the interaction between quoting decisions, inventory and configured constraints.

Control boundary

A strategy may propose an order; the risk layer decides whether the action is permitted. This separation mirrors an important production trading-system responsibility.

3.4 Position and P&L processing

Trades produced by matching are consumed by a P&L engine that maintains account cash, positions and valuation state. The workflow processes new fills, updates inventory, marks positions from the current book and produces a financial summary.

P&L tests use numerical tolerances and are kept separate from the direct latency timing loop. This shows how matching output feeds downstream financial state without allowing accounting and reporting work to distort engine measurements.

3.5 Sequenced UDP feed and recovery

The network simulation serialises events into UDP-style datagrams, submits them through a queue, decodes them and applies them through a recovery sequencer. Sequence numbers are treated as state-integrity controls: a later update cannot simply be accepted when an earlier update is missing.

A complete recovery source retains the ordered stream. When a gap is exposed, the missing event is recovered, applied in sequence and followed by the later buffered update. Feed-state transitions and recovery durations are recorded as telemetry.

3.6 Telemetry and analysis

The C++ system writes structured CSV telemetry for state, depth, quotes, fills, latency and recovery. Python analysis validates the result columns, aggregates repeated runs and produces the figures in this report. This creates an auditable path from execution to reported claim.

3.7 End-to-end processing flow

Stage	Primary output
LOBSTER loader	Normalised, sequenced MarketEvent stream
Binary codec	Deterministic serialised event file and verified round trip
Replay and order book	Price-time-priority market state
Strategy and risk	Candidate quotes filtered by configured controls
Matching	Trades and updated resting liquidity
P&L engine	Cash, inventory and marked account state
Telemetry and Python	Auditable CSV outputs and performance figures

4. Validation methodology

The project uses a layered validation programme. Each stage answers a different engineering question so that one successful benchmark is not treated as proof that every subsystem is correct.

Stage 1 - Core matching correctness

Tests establish price-time priority, FIFO behaviour, quantity arithmetic, cancellation, modification, aggressive matching, partial fills, completed-order removal and book invariants.

Stage 2 - Event and replay correctness

Normalised events are serialised and parsed through the binary codec. Round-trip equality checks ensure that storage and decoding preserve the logical event stream.

Stage 3 - Strategy and risk validation

The market-making workflow receives market state, generates candidate quotes and passes them through configured controls. Tests verify coherent strategy state and risk decisions.

Stage 4 - P&L and position validation

Trades update cash and inventory, positions are marked against the market, and numerical accounting outputs are checked with tolerances.

Stage 5 - UDP fault-recovery validation

A clean loss-free replay establishes the expected final state. The feed is then repeated with deliberate omissions. The recovered run must detect every gap, record zero misses, finish healthy, leave the queue empty, pass invariants and match the clean final book.

Stage 6 - Direct replay scalability

Five workload sizes are each executed five times. Source conversion, telemetry writing and post-batch invariant checking are outside the timed event-application interval. Latency is sampled once every 100 operations.

Stage 7 - UDP scalability

The same workload sizes are processed through datagram, queue, decode, sequencing and recovery stages, with one missing event injected approximately every 25,000 applications.

4.1 Direct replay timing boundary

- The LOBSTER CSV is loaded and converted before measurement.
- The source event vector is reused for workloads larger than 380,678 events.
- The order book is reset before each complete source pass to avoid duplicate order-ID collisions.
- Telemetry is disabled inside the measured replay path.
- Invariant validation occurs after each batch, outside the measured application interval.
- Latency is sampled rather than measured on every event to reduce timer overhead.
- Process memory is polled periodically and the peak observed working set is retained.

4.2 UDP correctness gates

- Every converted event must ultimately be applied.
- Detected gaps must equal deliberately omitted packets.
- Recovered events must equal omitted packets.
- Recovery misses, application rejections, decode failures and sequencer failures must remain zero.
- The final feed state must be Healthy.
- The processing queue must be empty.
- The final recovered book must equal the clean baseline book in the full recovery test.

4.3 Benchmark separation

Path	What it measures	What is excluded
Direct replay	Normalised MarketEvent application and active-book state changes	CSV parsing, telemetry writing and external networking
UDP scalability	Codec, queue, sequencing, gap detection, recovery and ordered application	Production NIC, multicast stack and remote recovery service
Risk/P&L tests	Control decisions, cash, inventory and mark-to-market accounting	Direct replay latency timing

5. Input data and conversion

Conversion metric	Count
Source rows	400,391
Converted events	380,678
Orders still active after conversion	10,085
Missing order references skipped	8,381
Hidden executions skipped	11,332
Control events ignored	0

The conversion totals reconcile exactly: $400,391 - 8,381 - 11,332 = 380,678$.

Missing references indicate that some messages refer to orders not present in the locally available starting state. Hidden executions cannot be mapped to a visible resting order. These rows are counted and skipped rather than being fabricated.

Data claim

The converted stream is suitable for engine-flow, correctness and scalability evaluation. It must not be described as a complete original-market L3 reconstruction.

5.1 Conversion policy

- Rows are normalised into one internal event representation before benchmarking.
- References to absent orders are counted and skipped rather than converted into invented state.
- Hidden executions are excluded from visible-order reconstruction.
- Conversion statistics are printed so the replayable population is explicit and reproducible.

6.1 Runtime scaling

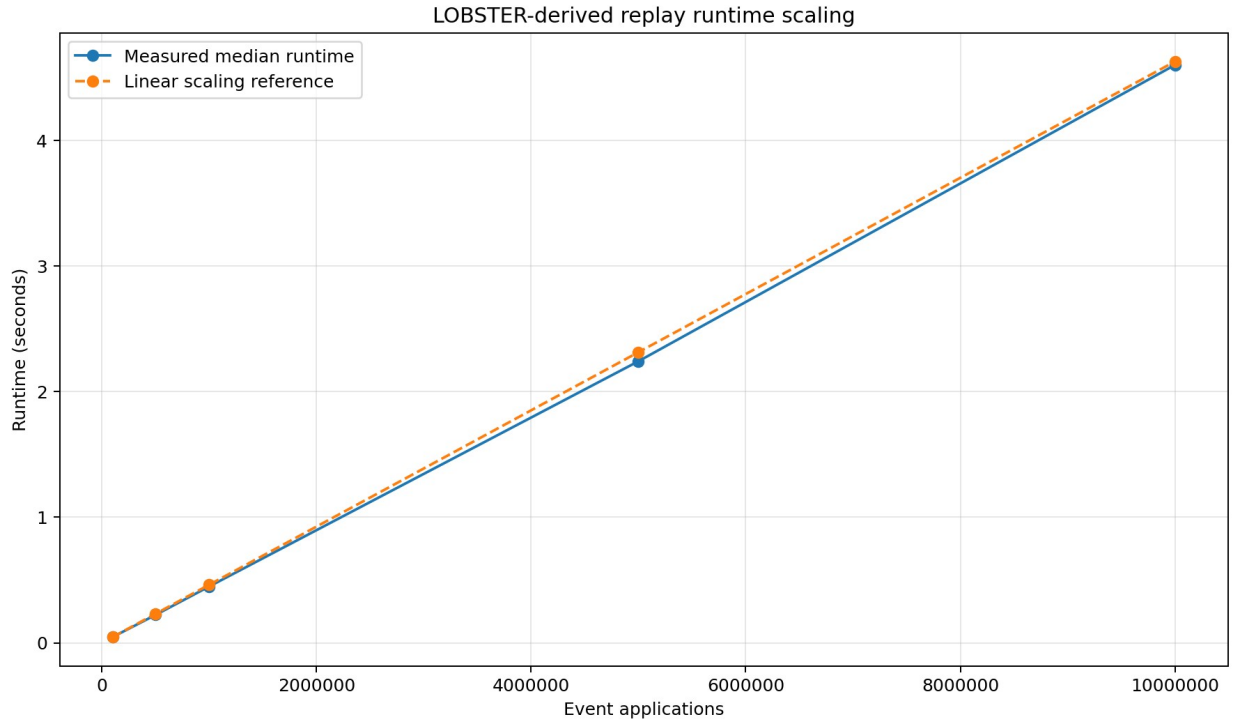


Figure 1. Median direct replay runtime compared with a linear scaling reference.

The measured median-runtime line follows the linear reference almost point for point. A 100-fold increase in workload, from 100,000 to 10 million event applications, increases median runtime from 0.046 seconds to 4.601 seconds - effectively the same 100-fold change. The small separation between the measured and reference lines remains consistent rather than widening as the workload grows.

The new finding is that the average cost per applied event remains broadly constant across the tested range. There is no visible scaling threshold at which order lookup, price-level maintenance or matching work begins to increase disproportionately. This supports approximately linear batch-replay behaviour for this dataset and active-book profile; because the source is replayed in bounded passes, the conclusion is limited to the tested state sizes.

6.2 Throughput stability

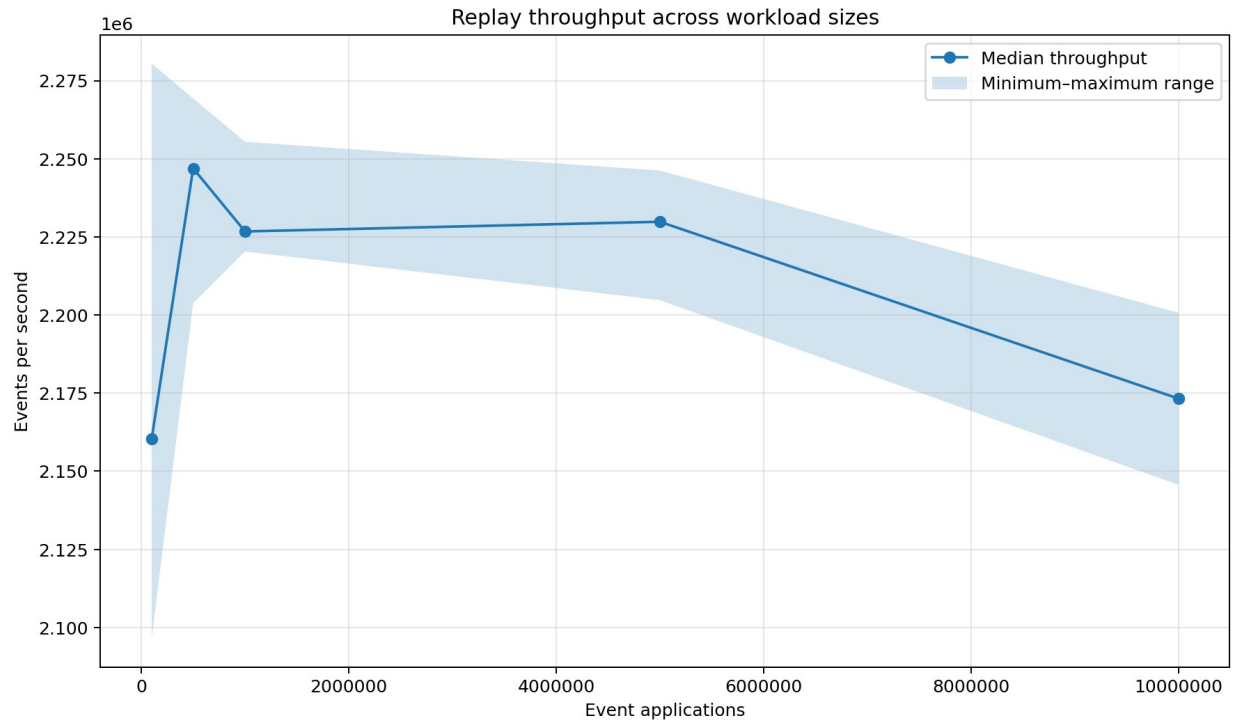


Figure 2. Median replay throughput and the observed minimum-to-maximum range across five repetitions.

Median throughput forms a plateau rather than declining in proportion to workload size. It rises from approximately 2.16 million events/s at 100,000 applications to a peak near 2.25 million at 500,000, remains around 2.23 million through 5 million, and finishes above 2.17 million at 10 million. The minimum-to-maximum band across five repetitions is relatively narrow, so the median is not driven by one unusually fast run.

This reveals that the engine reaches a steady operating rate once short-run fixed costs are amortised. The modest fall at 10 million is only a few percent from the mid-range plateau and indicates mild long-run system noise or cache/scheduling pressure rather than throughput collapse. The result demonstrates predictable capacity under increasing replay volume, although a controlled Release rerun is required before making final performance claims.

6.3 Sampled event latency

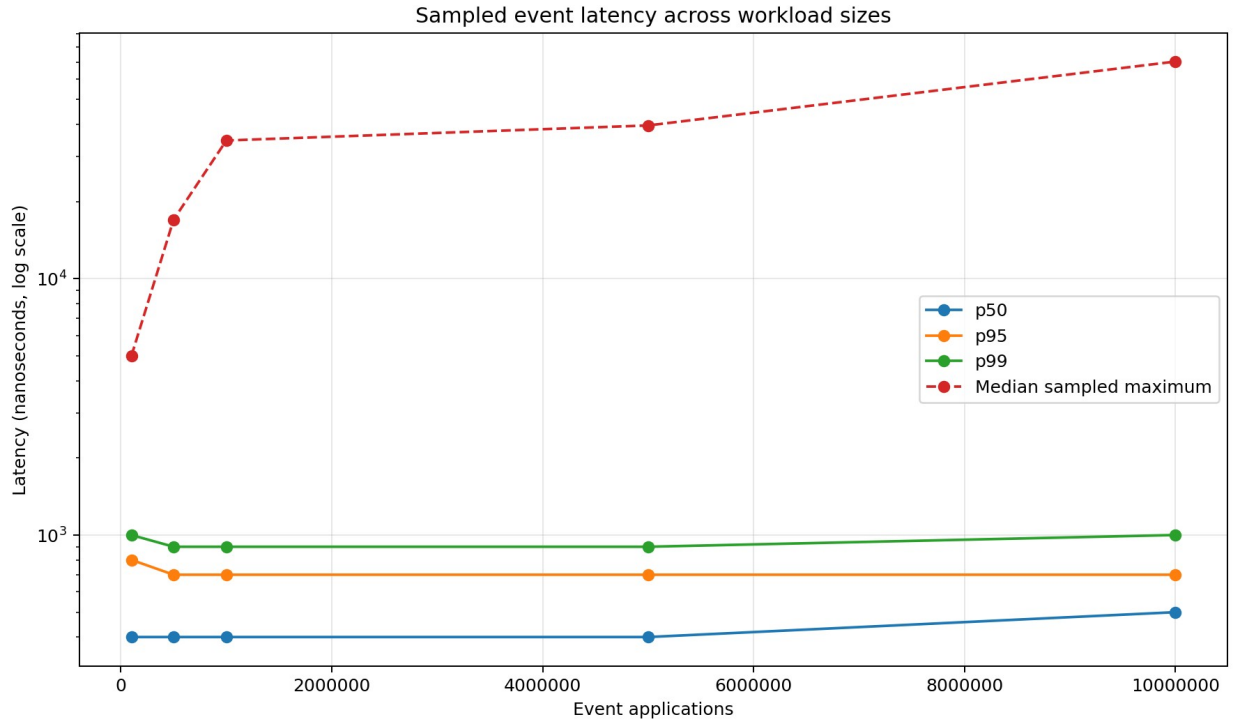


Figure 3. Sampled p50, p95, p99 and maximum direct-event latency.

The p50, p95 and p99 series are almost flat across the workload range. Median sampled latency stays around 0.4-0.5 microseconds, p95 around 0.7-0.8 microseconds and p99 around 0.9-1.0 microseconds, even when total work increases from 100,000 to 10 million applications. The routine event-application path therefore does not progressively slow during longer runs.

The sampled maximum rises while the percentile lines remain stable. That divergence shows that the larger observations are sparse outliers rather than a broad shift in the latency distribution. Larger workloads create more opportunities to capture scheduler pre-emption, cache disturbance or another platform interruption. The finding is stable tail-percentile behaviour with occasional system-level outliers, not deterministic worst-case latency.

6.4 Memory behaviour

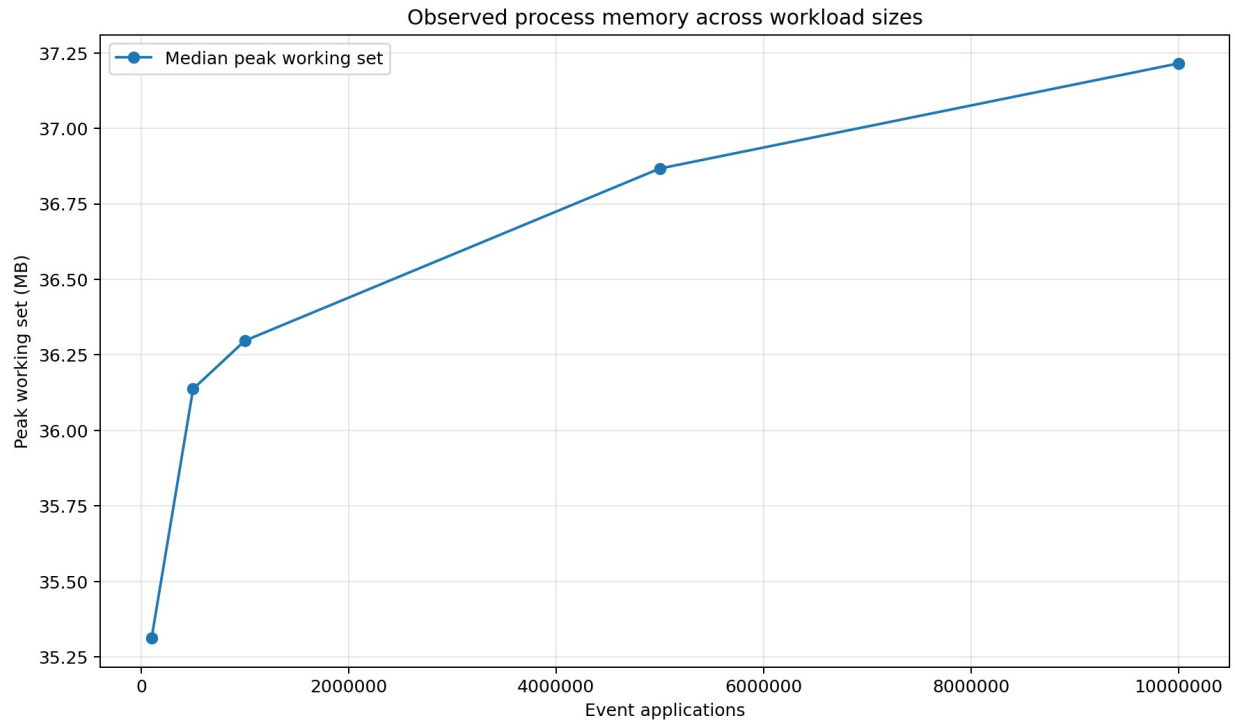


Figure 4. Median peak process working set across workload sizes.

The memory curve rises at smaller workloads and then progressively flattens. Median peak working set increases from 35.312 MB at 100,000 applications to 36.297 MB at 1 million, then reaches only 37.215 MB at 10 million. A 100-fold increase in event applications therefore produces an increase of approximately 1.903 MB, or 5.4%, rather than linear memory growth.

This separates fixed process/container overhead from workload-dependent state. Once the active book and order index have reached their operating size, additional replay volume does not cause historical events to accumulate indefinitely. The evidence is that resident memory is driven mainly by bounded active state, an important property for long-running market-data and simulation processes.

6.5 Final resting-order state

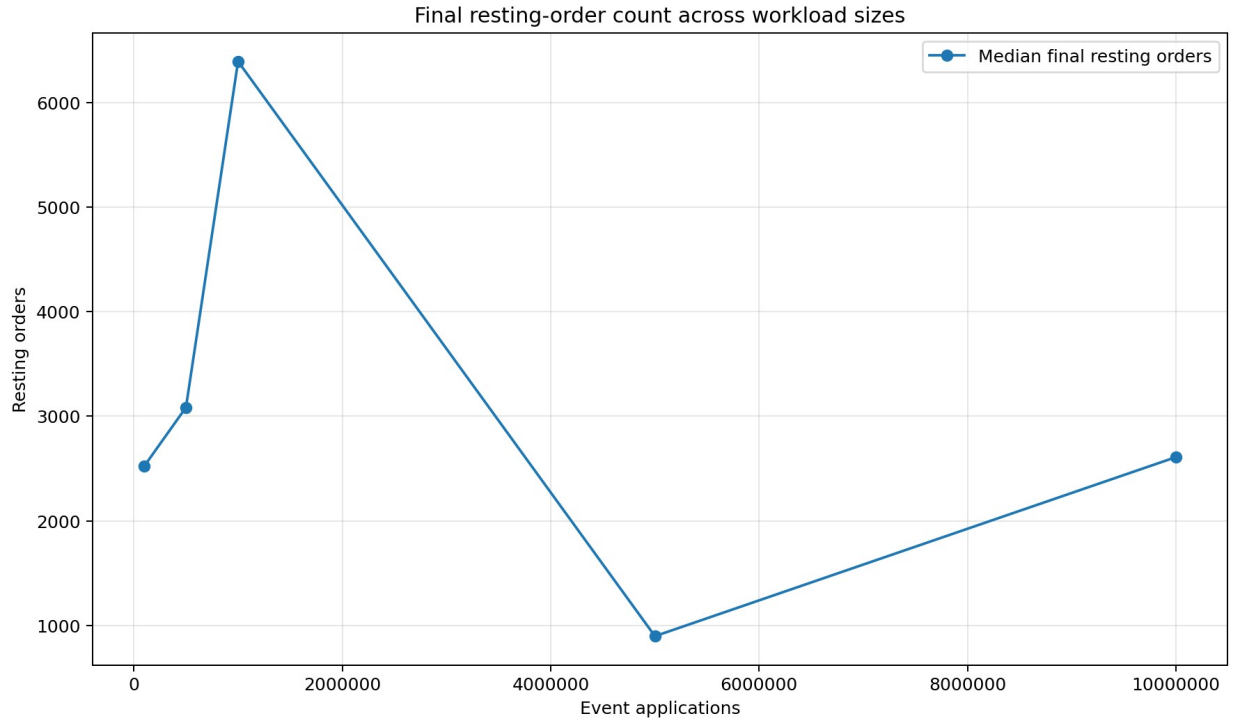


Figure 5. Median final resting-order count after each workload.

The final resting-order count rises to a peak at 1 million applications, falls sharply at 5 million and rises again at 10 million. This non-monotonic shape is expected: the 380,678-event source is repeated, the book is reset before each complete source pass, and every workload ends at a different prefix of its final pass.

The graph therefore shows that final book size depends on where replay stops in the event sequence, not on total events processed. Prefixes dominated by additions leave more resting orders, while prefixes containing more cancellations, deletions and executions leave fewer. It is a useful state-shape diagnostic and should not be interpreted as a memory-leak or scalability trend.

6.6 Correctness under load

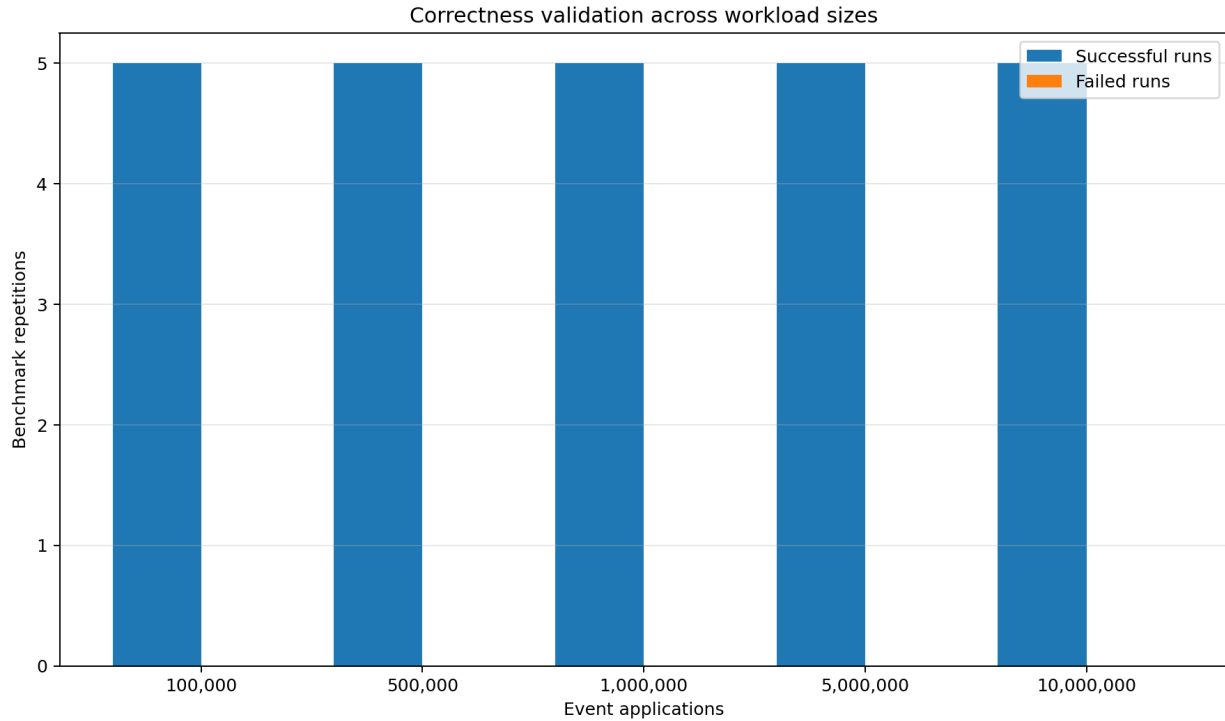


Figure 6. Successful and failed direct-replay repetitions at each workload size.

Every workload shows five successful repetitions and zero failed repetitions. Across all 25 direct-replay runs, no benchmark event was rejected and every post-run structural invariant check passed, including at 10 million event applications.

This turns correctness into a measured condition of the performance result. The latency and throughput figures are accepted only because the order index, price levels, queued orders and quantities remain internally consistent. The conclusion is stronger than process completion: the engine preserved defined state integrity under repeated multi-million-event workloads.

7. UDP sequencing and recovery results

The UDP benchmark measures a broader and deliberately more expensive path than direct replay. Each event can pass through serialisation, queueing, decoding, sequence validation, gap handling and recovery before ordered application.

7.0 Recovery workflow

Step	Behaviour
1. Encode	Convert a normalised event into a UDP-style datagram.
2. Queue	Submit the datagram to the network queue.
3. Decode	Validate and reconstruct the event payload.
4. Sequence	Compare the event sequence with the next expected sequence.
5. Detect gap	A later sequence exposes the deliberately omitted update.
6. Recover	Retrieve and apply the missing event from the complete recovery source.
7. Resume	Apply buffered later events in order and return the feed to Healthy.

Why this path is slower

The UDP benchmark intentionally measures resilience overhead that direct replay excludes. Lower throughput is expected because every event passes through additional protocol, queue and sequencing logic.

7.1 UDP processing throughput

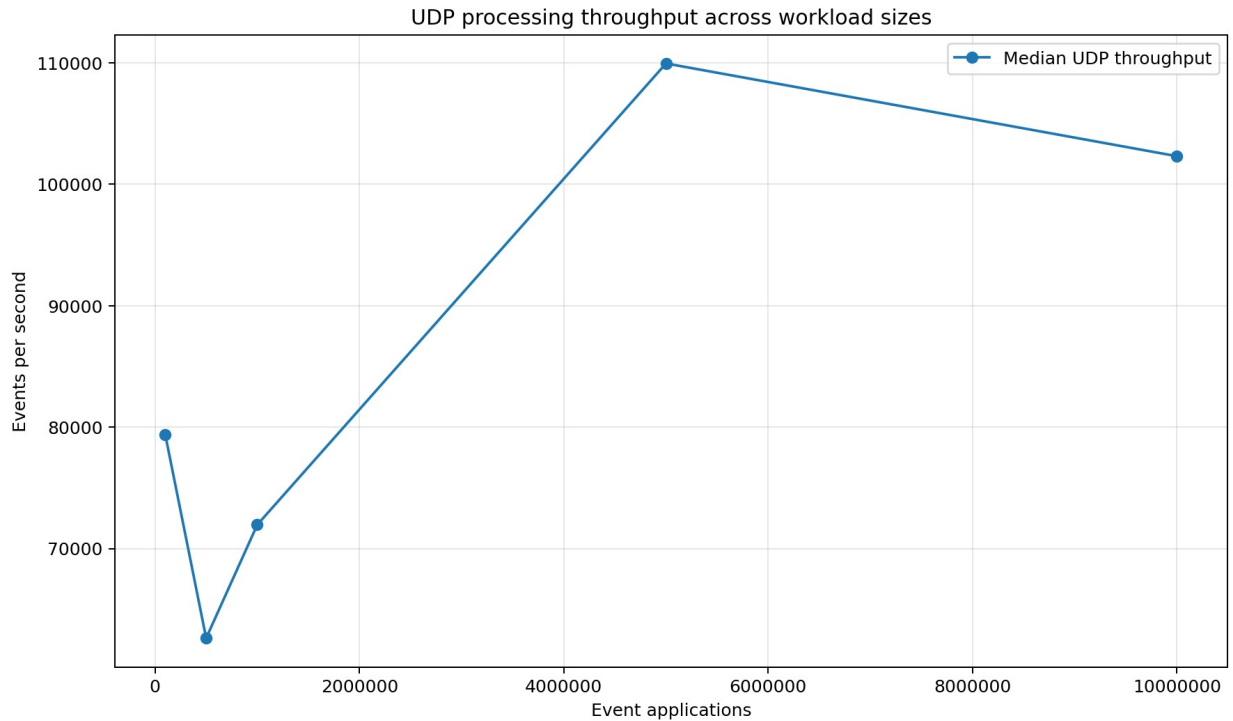


Figure 7. Median UDP processing throughput across workload sizes.

UDP throughput is non-monotonic at smaller workloads: it starts near 79,000 events/s, falls to roughly 63,000 at 500,000 applications, recovers to around 72,000 at 1 million, and then rises to approximately 110,000 at 5 million before settling near 102,000 at 10 million. This path performs serialisation, queueing, decoding, sequence checks, loss handling and recovery, so it is expected to operate well below direct replay.

At the larger workloads, throughput improves and remains above approximately 100,000 events/s, suggesting that fixed setup and per-pass costs are increasingly amortised. The current chart has no uncertainty band, so it should not be read as monotonic scaling. Its strongest contribution is to quantify resilience overhead: the broader ordered-recovery path is more than an order of magnitude slower than direct event application, making the cost of reliability explicit.

7.2 Recovery outcomes

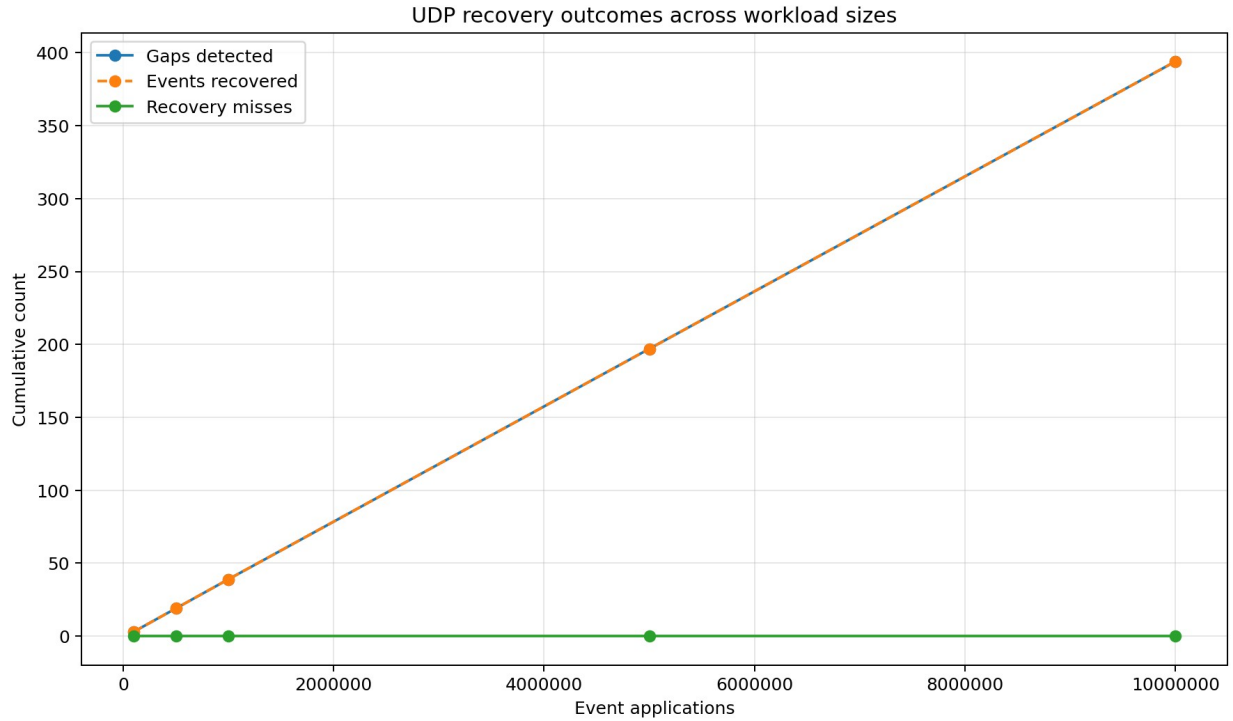


Figure 8. Detected gaps, recovered events and recovery misses.

Detected gaps and recovered events overlap at every workload size and increase almost linearly with total applications, while recovery misses remain fixed at zero. Because loss is injected at a regular interval, larger workloads create proportionally more incidents - reaching nearly 400 recoveries at 10 million applications - rather than repeating one hand-crafted case.

The new finding is that recovery correctness holds as both event volume and failure count increase. No divergence appears between detection and recovery, and no misses emerge at the largest workload. Together with baseline-state comparison and final invariant checks, this demonstrates repeated restoration of ordered state rather than merely logging that a sequence gap occurred.

7.3 Recovery latency

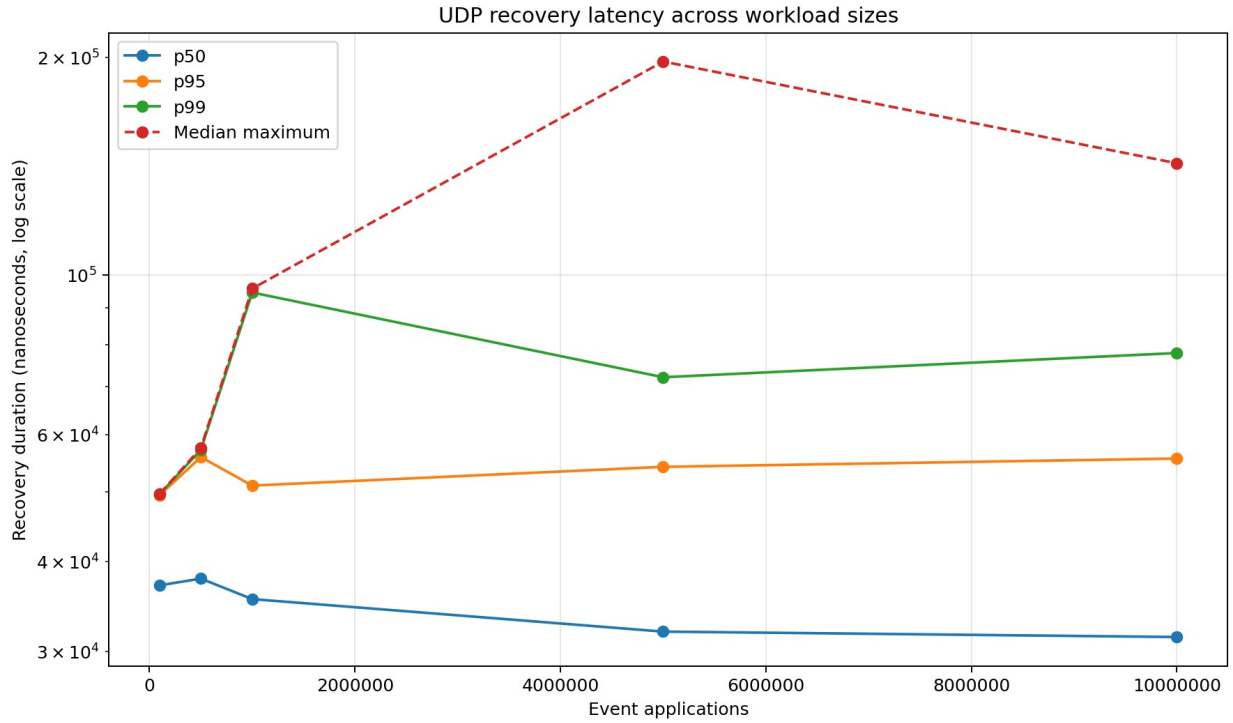


Figure 9. Recovery duration percentiles and median maximum on a logarithmic scale.

Median recovery duration remains within approximately 31-38 microseconds and trends slightly downward at larger workloads. The p95 series is similarly stable at roughly 49-57 microseconds. Neither percentile increases with total event volume, indicating that the sequencer does not accumulate a growing recovery backlog as the run becomes longer.

The p99 and median-maximum series vary more because recovery incidents are sparse and their tail estimates are more sensitive to Windows scheduling and isolated pauses. The sound conclusion is that typical recovery cost remains bounded while rare tails remain platform-sensitive. More importantly, every injected gap is recovered, misses remain at zero and the feed returns to a healthy ordered state.

8. Why the broader components matter

Matching without risk is incomplete - A matching engine can execute orders correctly while still allowing a strategy to create unsafe exposure. Separating candidate quote generation from risk approval reflects a critical control boundary in electronic trading.

Trades without accounting are incomplete - A fill changes inventory, cash and marked value. The P&L engine demonstrates how matching output is consumed by downstream financial state and why accounting requires independent numerical validation.

Fast processing without recovery is fragile - A book may remain structurally consistent while becoming economically wrong if later updates are applied after a lost packet. Fault-injection and baseline state comparison address that failure mode directly.

Results without telemetry are difficult to audit - Structured CSV output and deterministic chart generation allow reviewers to inspect raw results, rerun the analysis and trace headline claims back to recorded data.

9. Relevance to financial engineering roles

Exchanges and trading venues

Demonstrates price-time priority, deterministic state transitions, sequence integrity, tail-latency measurement, state recovery and correctness validation under load.

Banks, brokers and electronic-execution teams

Demonstrates replay for investigation, separation of strategy and risk, position/P&L integration, ordered market-data handling and measurable recovery behaviour.

Systematic funds and quantitative platforms

Demonstrates event-driven research infrastructure, deterministic replay, large-batch processing, bounded memory, auditable telemetry and Python-based analysis.

10. Limitations and interpretation boundaries

- Results were generated from a Debug build.
- The tests ran on a general-purpose Windows machine.
- Direct replay excludes CSV parsing, file I/O and external network transport.
- Latency is sampled once every 100 applications.
- Workloads above 380,678 reuse the converted source stream.
- The available data does not support a complete original-market L3 reconstruction.
- Only one AAPL-derived interval was evaluated.
- UDP recovery is an in-process simulation, not a production multicast or NIC benchmark.
- Risk and P&L are functionally validated but are not included in the direct replay timed region.

Why this strengthens the report

These boundaries make the claims precise. They show awareness of benchmark design and prevent development measurements from being overstated as production exchange performance.

11. Recommended next steps

1. Repeat the complete suite in Release mode.
2. Record CPU, RAM, operating-system and compiler metadata with every run.
3. Increase repetitions and report confidence intervals and variance.
4. Pin the benchmark process to a dedicated core and control the power plan.
5. Compare MinGW GCC and MSVC builds.
6. Profile allocations, branches and cache misses.
7. Add isolated cost breakdowns for codec, queue, sequencer and recovery.
8. Add per-operation latency distributions for add, cancel, modify and execute.
9. Test additional symbols, trading periods and book shapes.
10. Extend fuzzing, property-based tests and sanitiser coverage.
11. Add richer risk scenarios and portfolio-level P&L tests.
12. Seed the initial book when a suitable snapshot is available.

12. Conclusion

The project demonstrates more than a functioning order book. It establishes a connected evidence chain from real event-flow conversion, through deterministic matching and replay, to risk/P&L integration, fault-injected recovery, correctness-gated benchmarking and reproducible analysis.

Within the measured Debug environment, the direct engine scales approximately linearly to 10 million event applications, maintains stable multi-million-event throughput, preserves sampled sub-microsecond p99 event latency and shows controlled memory growth. All direct runs pass invariant checks with zero rejected events. The UDP path detects and recovers every deliberately omitted event with zero recovery misses.

Portfolio value

A documented Release-build rerun would strengthen the quantitative claims, but the repository already demonstrates the engineering concerns expected in exchange technology, electronic trading, market-data infrastructure and quantitative-platform roles: ordered state, deterministic replay, risk separation, financial accounting, resilience and measurable correctness under load.